Image Classification using Supervised CNNs and Self-Supervised Representation Learning

Nawroz Mohammadi
ID: 947798
Universit‘ a degli Studi di Milano-Bicocca

PALLAB MONDAL
ID: 946804
Universit‘ a degli Studi di Milano-Bicocca

***Abstract*—This project investigates supervised and self-supervised learning approaches for food image classification under limited data availability and computational constraints. The main objective is to evaluate whether self-supervised representation learning can provide more effective visual features compared with conventional supervised training when only a limited amount of labeled data is available.**

A food image dataset containing 251 categories was used, where only 30% of the available data was selected while preserving the original class distribution. A ResNet-inspired Convolutional Neural Network (CNN) was trained as a supervised baseline using practical optimization techniques, including data augmentation, class imbalance handling, learning-rate scheduling, Automatic Mixed Precision, and GPU memory optimization strategies.

A Self-Supervised Learning approach based on SimCLR was then investigated, where the same CNN encoder was trained using contrastive learning on augmented image pairs without relying on labels. The learned representations were transferred to a classification task through feature extraction and fine-tuning.

The experiments highlight the challenges of training deep learning models with limited labeled data and highly imbalanced classes. While supervised learning provides a simpler and more computationally efficient solution, self-supervised pretraining demonstrates the potential to learn more robust visual representations. Overall, this study emphasizes the trade-off between computational cost and representation quality when applying deep learning techniques under resource constraints.

***Index Terms*—Image Classification using Supervised CNNs and Self-Supervised Representation Learning**

I. Introduction

A. Problem Statement

Image classification is one of the fundamental tasks in computer vision, where deep learning models are used to automatically recognize and categorize visual information. In this project, the task focuses on food image classification across 251 different categories.

However, training accurate classification models becomes challenging when working with limited labeled data, highly imbalanced classes, and restricted computational resources. Some food categories contain significantly fewer samples than others, making it difficult for models to learn balanced representations. Additionally, GPU memory limitations require careful consideration of model complexity and training strategies.

Therefore, this project investigates practical approaches for training image classification models under realistic constraints while comparing conventional supervised learning with self-supervised representation learning.

B. Motivation

Deep learning models often rely on large labeled datasets, which can be difficult to obtain. This project investigates whether self-supervised learning can improve feature representation when labeled data is limited, while considering practical computational constraints.

C. Project Objective

The objective of this project is to compare a conventional supervised CNN approach with a SimCLR-based self-supervised learning approach for food image classification. The study evaluates their ability to learn useful representations using a limited and imbalanced dataset.

D. Main Contributions

The main contributions of this project are:

- Development of a CNN-based supervised classification pipeline under limited data and GPU constraints.
- Investigation of practical optimization techniques, including data augmentation, imbalance handling, and efficient training strategies.
- Evaluation of self-supervised representation learning using SimCLR and comparison with supervised training.

II. Dataset and Experimental Setup

A. Dataset Description

The experiments were conducted using a food image classification dataset containing 251 different food categories. The original dataset consists of 118,475 training images and 11,994 testing images.

The dataset presents a challenging classification problem due to significant class imbalance, where some categories contain considerably fewer samples than others. Additionally, images have different resolutions and visual characteristics, requiring preprocessing before training.

B. Dataset Sampling Strategy

Due to computational limitations, only 30% of the original dataset was randomly selected for training. The sampling process was performed while preserving the original class distribution to maintain the characteristics of the full dataset.

A fixed random seed was used during sampling to ensure reproducibility and allow a fair comparison between supervised and self-supervised learning approaches.

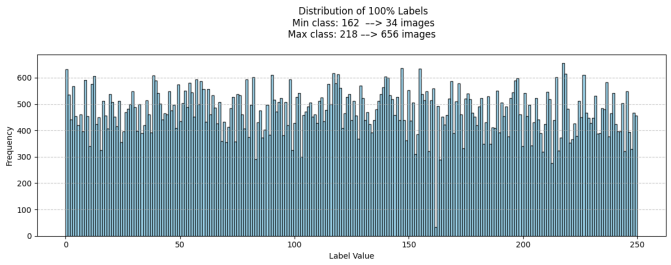

Fig. 1. Distribution of images across the food categories in the original dataset.

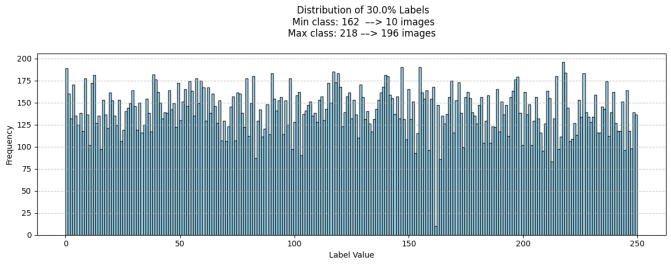

Fig. 2. Comparison between the original dataset distribution and the selected 30% subset.

C. Data Splitting

After dataset sampling, the selected images were divided into training, validation, and testing sets. The training set was used for model optimization, while the validation set was used to monitor performance during training and adjust training decisions. The test set was reserved for the final evaluation of the trained model.

The final dataset distribution was:

Dataset Split	Number of Images
Training	26,568
Validation	8,856
Testing	3,485

TABLE I
FINAL DATASET SPLIT USED FOR TRAINING AND EVALUATION.

The validation set was created from the sampled training data using a 75%/25% split. The same dataset split was maintained for both supervised and self-supervised experiments to ensure a fair comparison.

D. Hardware and Software Environment

The experiments were conducted using a GPU-accelerated environment to enable efficient training of deep learning models. The implementation was developed using the PyTorch framework with CUDA support for GPU computation.

The main software components included Python, PyTorch, CUDA, and additional libraries for data processing, model training, and evaluation.

The training pipeline also incorporated GPU optimization techniques such as Automatic Mixed Precision (AMP) and efficient data loading to reduce memory consumption and improve training speed.

Component	Description
Programming Language	Python
Deep Learning Framework	PyTorch
GPU Acceleration	CUDA
Training Optimization	AMP, torch.compile
Data Processing	PyTorch Dataset/DataLoader

TABLE II
SOFTWARE ENVIRONMENT USED FOR MODEL DEVELOPMENT AND TRAINING.

E. Evaluation Metrics

The performance of the classification models was evaluated using multiple metrics to provide a more complete analysis, especially considering the class imbalance in the dataset.

Accuracy was used to monitor the overall classification performance during training. Additionally, Precision, Recall, and F1-score were calculated during final evaluation to measure performance across different classes.

A confusion matrix was also used to analyze classification errors and identify categories that were more difficult to distinguish.

The evaluation metrics are defined as:

- **Accuracy:** Measures the percentage of correctly classified samples.
- **Precision:** Measures the proportion of correct predictions among all predicted samples for a class.
- **Recall:** Measures the proportion of correctly identified samples among all actual samples of a class.
- **F1-score:** Represents the harmonic mean of precision and recall.

Metric	Purpose
Accuracy	Overall classification performance
Precision	Correctness of positive predictions
Recall	Ability to identify samples correctly
F1-score	Balance between precision and recall
Confusion Matrix	Class-level error analysis

TABLE III
EVALUATION METRICS USED IN THE EXPERIMENTS.

III. SUPERVISED CNN APPROACH

A. CNN Architecture

A custom CNN architecture inspired by the ResNet family was implemented as the supervised learning baseline. The model uses residual connections to improve gradient flow and enable stable training.

The architecture consists of five residual blocks followed by a classification head with 251 output neurons. Batch Normalization and ReLU activation functions are applied throughout the network, with dropout used before the final classifier to reduce overfitting.

The final model contains approximately 6.47 million trainable parameters, providing a balance between representation capability and computational efficiency under GPU memory constraints.

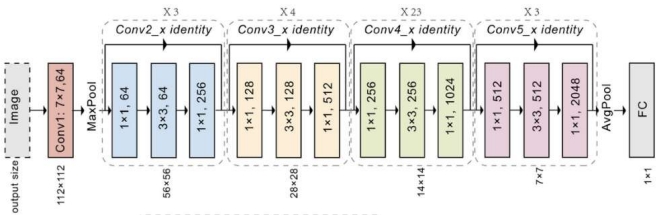

Fig. 3. Overview of the ResNet-inspired CNN architecture used for supervised classification.

B. Data Preprocessing

Before training, all images were preprocessed to provide consistent input to the CNN model. Images were resized to 160×160 pixels to reduce computational cost while preserving relevant visual information.

Dataset normalization was applied using the calculated mean and standard deviation values to stabilize training. During training, data augmentation techniques, including random cropping, brightness adjustment, contrast variation, and horizontal flipping, were applied to improve model generalization.

These preprocessing steps were selected to improve training efficiency and reduce overfitting when using a limited subset of the dataset.

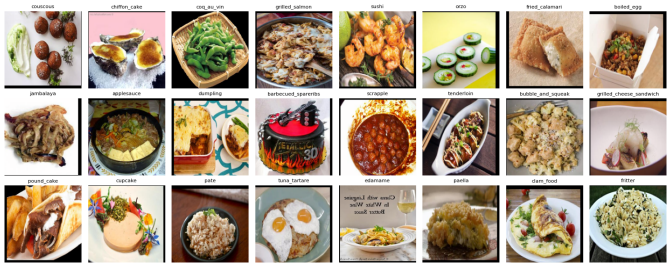

Fig. 4. Image preprocessing pipeline used before CNN training

C. Class Imbalance Handling

The dataset contains significant differences in the number of samples between food categories, which can bias the model toward majority classes.

To reduce this effect, class frequencies were calculated and used to create a weighted sampling strategy. The `WeightedRandomSampler` was applied during training to increase the probability of selecting samples from minority classes without modifying the original dataset.

This approach helps the model receive a more balanced representation of classes during training.

D. Training Strategy

The CNN was trained for 50 epochs using the AdamW optimizer and Cross-Entropy Loss for multi-class classification. A learning-rate scheduler was applied to gradually reduce the learning rate during training and improve convergence.

Automatic Mixed Precision (AMP) was used to reduce GPU memory consumption and accelerate training while maintaining numerical stability. Early stopping was also implemented to monitor validation performance, although the stopping condition was not reached during training.

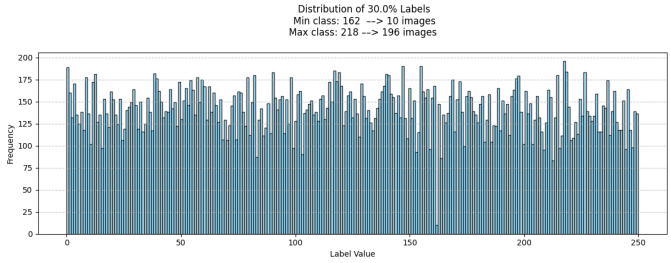

Fig. 5. Class imbalance in the dataset.

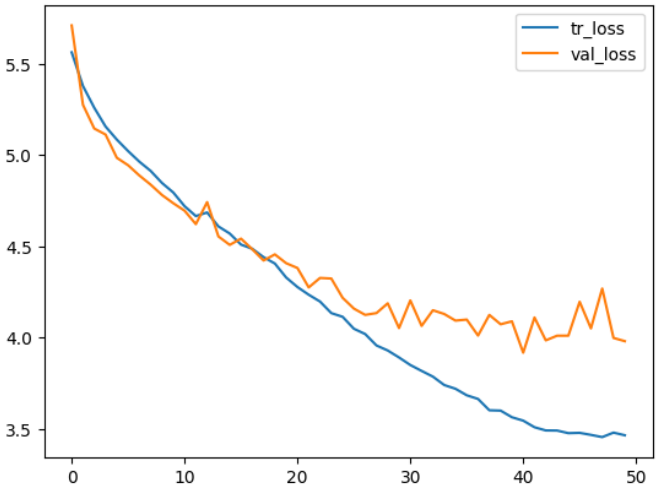

Fig. 6. Training and validation loss during supervised CNN training.

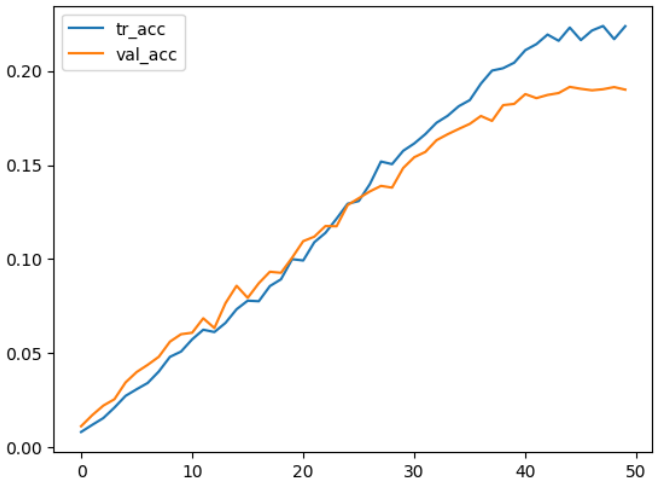

Fig. 7. Training and validation accuracy during supervised CNN training.

E. Computational Optimization Techniques

Due to GPU memory limitations, several optimization techniques were applied to improve training efficiency. Automatic Mixed Precision (AMP) was used to reduce memory usage

and accelerate GPU computation while maintaining training stability.

Additionally, efficient data loading strategies, including pinned memory, were applied to improve CPU-to-GPU data transfer. The model size was also controlled by limiting the number of parameters to approximately 6.47 million, providing a balance between performance and computational cost.

F. Supervised CNN Results

The supervised CNN was evaluated using training and validation performance during 50 epochs. The model achieved a final training accuracy of 22.38% and validation accuracy of 19.00%.

The final evaluation was further performed using Precision, Recall, and F1-score due to the strong class imbalance in the dataset. The obtained validation results were:

- Precision: 0.1647
- Recall: 0.1898
- F1-score: 0.1668

The test evaluation achieved:

- Precision: 0.1917
- Recall: 0.2095
- F1-score: 0.1854

The results show that the model successfully learned meaningful visual patterns; however, performance was limited by the reduced dataset size and significant class imbalance.

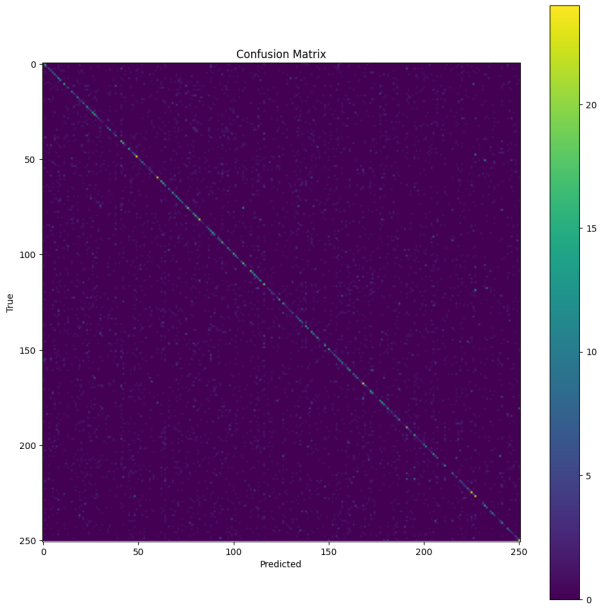

Fig. 8. Confusion matrix of the supervised CNN evaluation

G. Discussion

The supervised CNN successfully learned meaningful visual features from the food dataset despite the limited training data and strong class imbalance. The continuous decrease in training loss indicates effective optimization; however, the slower improvement of validation performance suggests that generalization was limited.

The results highlight the difficulty of fine-grained food classification when some categories contain fewer samples than others. Although weighted sampling and data augmentation helped reduce the impact of imbalance, minority classes remained challenging to classify.

Overall, the supervised CNN provided a reliable baseline while demonstrating the importance of practical techniques such as augmentation, imbalance handling, and GPU-efficient training when developing models under computational constraints.

IV. Self-Supervised Learning Approach (SimCLR)

A. Motivation

The supervised CNN approach requires labeled images for learning, which can become challenging when labeled data is limited or expensive to obtain.

To investigate alternative learning strategies, a Self-Supervised Learning approach based on SimCLR was explored. The objective was to evaluate whether visual representations learned without labels can improve downstream classification performance under limited data conditions.

B. SimCLR Framework

SimCLR is a Self-Supervised Learning framework that learns visual representations by comparing different augmented views of the same image. Instead of using class labels, the model learns features by maximizing the similarity between positive image pairs and separating different images.

The implemented SimCLR model consists of a ResNet-based encoder followed by a projection head that maps extracted features into a 128-dimensional embedding space. The encoder is later used for downstream food classification.

The total architecture contains approximately 6.63 million parameters, maintaining computational efficiency while enabling representation learning.

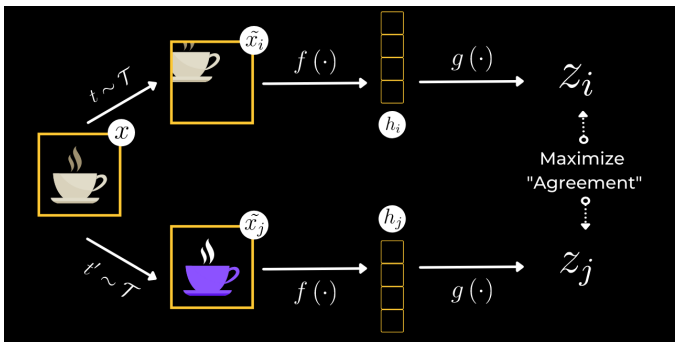

Fig. 9. SimCLR framework.

C. SSL Data Augmentation

In SimCLR, data augmentation is not only used for regularization but also provides the main learning signal. Each image is transformed into two different augmented views, which are treated as a positive pair during contrastive learning.

The augmentation pipeline included resizing, random cropping, horizontal flipping, rotation, color variation, and Gaussian blur. These transformations encourage the encoder to learn robust visual features that remain consistent under different image appearances.

D. Contrastive Learning Training

During SimCLR pretraining, the model receives two augmented views of the same image instead of image-label pairs. The encoder learns representations by maximizing similarity between positive pairs and separating representations from different images.

The NT-Xent (Normalized Temperature-scaled Cross Entropy) loss was used as the contrastive objective during SimCLR pretraining. Unlike supervised classification losses, NT-Xent does not require class labels. Instead, it learns visual representations by maximizing the similarity between two augmented views of the same image while minimizing the similarity between representations from different images.

Given a batch containing augmented image pairs, the encoder produces normalized feature representations z_i and z_j. The similarity between two representations is measured using cosine similarity:

$$sim(z_i, z_j) = \frac{z_i^T z_j}{||z_i|| ||z_j||} \tag{1}$$

The NT-Xent loss for a positive pair is defined as:

$$\mathcal{L}_{i,j} = -\log \frac{\exp(sim(z_i, z_j)/\tau)}{\sum_{k=1}^{2N} \mathbb{1}_{[k \neq i]} \exp(sim(z_i, z_k)/\tau)} \tag{2}$$

where N represents the batch size, z_j is the positive example corresponding to z_i, and τ is the temperature parameter controlling the concentration of similarity values.

The loss encourages positive pairs generated from the same image to have similar embeddings while pushing representations of different images apart. Through this objective, the encoder learns general visual features without requiring manual labels.

The model was trained using the same dataset subset as the supervised baseline to ensure a fair comparison between learning strategies.

E. Computational Optimization

Due to the higher computational cost of contrastive learning, several optimization techniques were applied during SimCLR training. Automatic Mixed Precision (AMP) was used to reduce GPU memory usage and accelerate training.

Additionally, PyTorch compilation was applied to optimize model execution. These techniques allowed the model to train efficiently while handling the increased computation caused by generating and processing two augmented views per image.

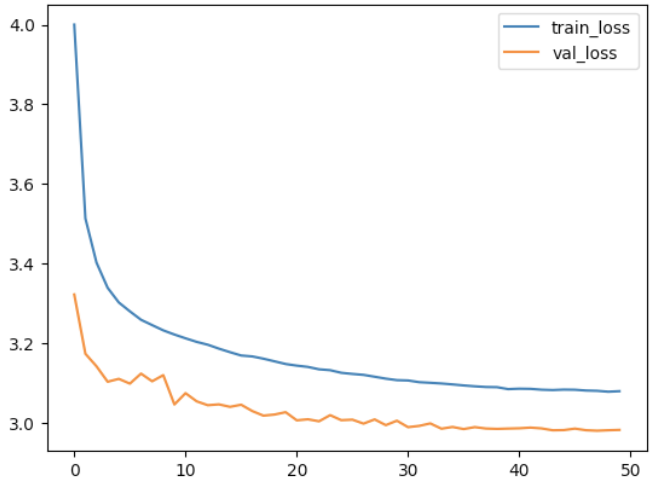

Fig. 10. Training and validation contrastive loss during SimCLR pretraining

F. Feature Extraction and Classification

After SimCLR pretraining, the learned encoder was transferred to a supervised classification task. The encoder was used as a feature extractor, while a new classification head was added for predicting the 251 food categories.

To adapt the pretrained representations to the downstream task, higher-level encoder layers were fine-tuned while early layers remained frozen. This strategy reduces training cost while preserving the general visual features learned during self-supervised pretraining.

The classifier was trained using Cross-Entropy Loss and evaluated using the same dataset split as the supervised CNN baseline.

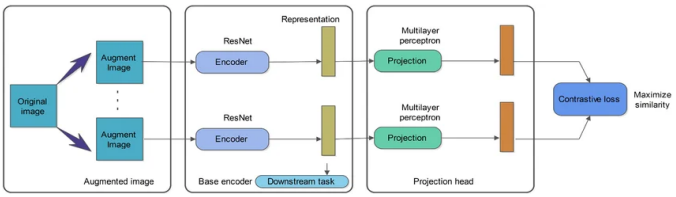

Fig. 11. Transfer learning pipeline using the SimCLR pretrained encoder for food classification.

G. SimCLR Results

The SimCLR model was first trained using contrastive learning to learn visual representations without labels. The model was then fine-tuned for food classification using the pretrained encoder.

During pretraining, the contrastive loss continuously decreased, indicating that the encoder learned increasingly meaningful image representations.

After fine-tuning, the model achieved:

- Training Accuracy: 18.51%
- Validation Accuracy: 14.42%

The results show that the self-supervised approach was able to learn useful features from unlabeled image transformations.

However, the final classification performance was still affected by the limited dataset size and high number of food categories.

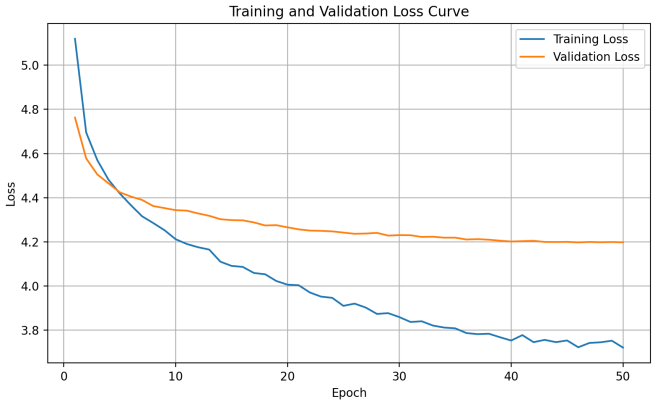

Fig. 12. Training and validation contrastive loss during SimCLR pretraining.

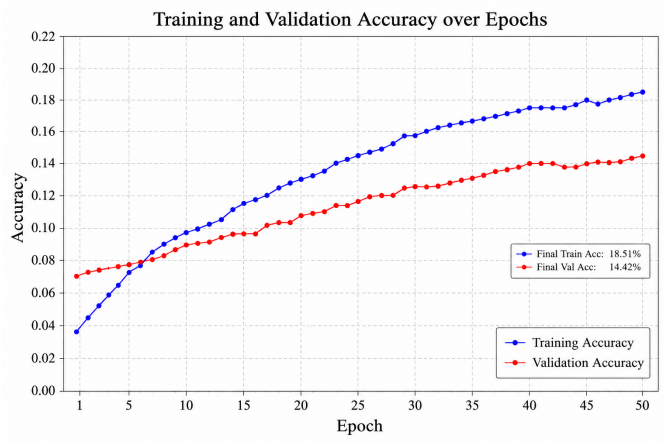

Fig. 13. Training and validation performance during SimCLR fine-tuning for classification.

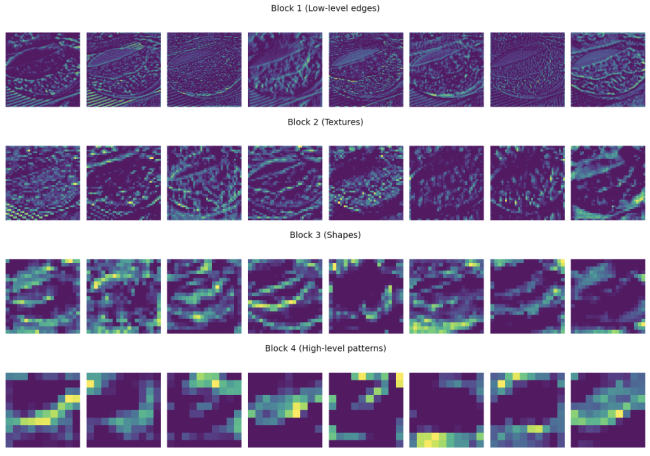

Fig. 14. Feature maps SimCLR Encoder layers learned

H. Discussion

The SimCLR experiment demonstrated that self-supervised pretraining can learn useful visual representations without relying on class labels. The pretrained encoder was able to transfer learned features to the downstream classification task.

Compared with the supervised CNN baseline, the SimCLR approach required more computational resources due to the additional pretraining stage and contrastive loss calculation. However, it provided a more general feature representation and reduced the dependency on labeled data.

The results highlight the trade-off between training efficiency and representation quality. While supervised learning offers a simpler and faster pipeline, self-supervised learning provides a promising alternative when labeled data is limited.

V. Comparison Between Supervised CNN and Self-Supervised Learning

A. Performance Comparison

The supervised CNN and SimCLR-based approaches were evaluated under the same dataset conditions using the same 30% sampled dataset and evaluation split.

The supervised CNN provides a direct end-to-end classification pipeline, while the SimCLR approach first learns visual representations and then transfers these features to a classification task.

Method	Training Accuracy	Validation Accuracy
Supervised CNN	22.38%	19.00%
SimCLR + Fine-tuning	18.51%	14.42%

TABLE IV
Comparison of supervised CNN and SimCLR-based classification performance.

The supervised CNN achieved higher final classification accuracy, mainly because it directly optimized the classification objective using labeled data. However, the SimCLR model focused on learning more general visual representations before classification.

B. Representation Learning Analysis

The main difference between the two approaches is the way visual features are learned. The supervised CNN learns features directly from labeled images, optimizing representations specifically for the classification task.

In contrast, SimCLR learns general visual representations without using labels by comparing different augmented views of the same image. This allows the encoder to capture more general patterns such as textures, shapes, and visual similarities.

Although the supervised CNN achieved higher classification accuracy, the SimCLR approach demonstrated the ability to learn useful features from unlabeled data, which can be beneficial when labeled samples are limited.

C. Computational Cost Comparison

The two approaches showed different computational requirements. The supervised CNN required only a single training stage, making it faster and more memory efficient.

In contrast, the SimCLR approach required an additional self-supervised pretraining phase, where two augmented views

of each image were processed and compared using contrastive loss. This increased training time and GPU memory requirements.

Although SimCLR introduced additional computational cost, it provided a way to learn useful representations without relying entirely on labeled data.

Method	Training Stages	Computational Cost
Supervised CNN	Classification only	Lower
SimCLR + Fine-tuning	Pretraining + Classification	Higher

TABLE V
COMPARISON OF COMPUTATIONAL REQUIREMENTS BETWEEN THE TWO APPROACHES.

D. Overall Discussion

The comparison between supervised CNN training and SimCLR-based learning highlights the different advantages of each approach.

The supervised CNN achieved better classification accuracy because it directly optimized the final classification objective using labeled data. However, its performance was limited by the amount of labeled data and class imbalance.

The SimCLR approach required more computational resources but demonstrated the ability to learn meaningful representations without using labels during pretraining. This makes self-supervised learning a promising strategy when labeled data is limited.

Overall, the experiments show that the choice between supervised and self-supervised learning depends on the available resources, annotation availability, and the target application.

VI. CONCLUSION

A. Summary of Findings

This project investigated food image classification using two different learning strategies: supervised CNN training and Self-Supervised Learning with SimCLR.

The supervised CNN provided a strong baseline by combining practical techniques such as data augmentation, class imbalance handling, learning-rate scheduling, and GPU-efficient training. The SimCLR approach demonstrated that useful visual representations can be learned without labels and transferred to downstream classification.

The experiments highlight the trade-off between computational efficiency and representation quality when working with limited labeled data and constrained resources.

B. Lessons Learned

The experiments demonstrated that model performance depends not only on architecture design but also on effective data preparation and training strategies.

Handling class imbalance, applying suitable augmentation, optimizing GPU usage, and selecting appropriate learning strategies were important factors for achieving stable training.

The project also showed that Self-Supervised Learning can provide valuable feature representations, especially when labeled data is limited, although it requires additional computational resources.

C. Future Improvements

Several improvements could be explored in future work. Training with the full dataset could provide more examples for minority classes and improve classification performance.

Further experiments could investigate stronger augmentation methods, alternative loss functions for imbalanced classification, larger pretrained architectures, and more extensive hyperparameter optimization.

For the Self-Supervised Learning approach, additional experiments with larger unlabeled datasets and different contrastive learning strategies could further improve feature quality.

REFERENCES

[1] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016. Available: https://arxiv.org/abs/1512.03385

[2] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A Simple Framework for Contrastive Learning of Visual Representations," in *International Conference on Machine Learning (ICML)*, 2020. Available: https://arxiv.org/abs/2002.05709

[3] S. Ioffe and C. Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," in *International Conference on Machine Learning (ICML)*, 2015. Available: https://arxiv.org/abs/1502.03167

[4] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A Simple Way to Prevent Neural Networks from Overfitting," *Journal of Machine Learning Research*, vol. 15, pp. 1929–1958, 2014. Available: https://jmlr.org/papers/v15/srivastava14a.html

[5] I. Loshchilov and F. Hutter, "Decoupled Weight Decay Regularization," in *International Conference on Learning Representations (ICLR)*, 2019. Available: https://arxiv.org/abs/1711.05101

[6] P. Micikevicius et al., "Mixed Precision Training," in *International Conference on Learning Representations (ICLR)*, 2018. Available: https://arxiv.org/abs/1710.03740

[7] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2012. Available: https://papers.nips.cc/paper/4824-imagenet-classification-with-deep-convolutional-neural-networks

[8] C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.

[9] A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019. Available: https://arxiv.org/abs/1912.01703

[10] I. Loshchilov and F. Hutter, "SGDR: Stochastic Gradient Descent with Warm Restarts," in *International Conference on Learning Representations (ICLR)*, 2017. Available: https://arxiv.org/abs/1608.03983

[11] Y. Cui, M. Jia, T. Lin, Y. Song, and S. Belongie, "Class-Balanced Loss Based on Effective Number of Samples," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019. Available: https://arxiv.org/abs/1901.05555

[12] D. M. W. Powers, "Evaluation: From Precision, Recall and F-Measure to ROC, Informedness, Markedness and Correlation," *Journal of Machine Learning Technologies*, vol. 2, no. 1, pp. 37–63, 2011. Available: https://arxiv.org/abs/2010.16061